PythonEX — Full Course

UNIT 1 45 min

Your First Python Program

Print output, store values in variables, write comments, and read error messages.

By the end of this unit you will be able to

- Run a Python statement and explain what `print()` does
- Create variables with the assignment operator `=`
- Follow the naming rules for variables and choose readable names
- Write comments that explain intent
- Read a simple error message and find the line that caused it

Why Python

Python reads close to plain English, which makes it a good first language and a strong tool for working professionals. The same language runs web sites, data analysis, automation, and machine-learning models. Everything you learn in this course applies to all of those.

Your first program

A Python program is a list of **statements**. Python runs them from top to bottom, one at a time. The `print()` function displays a value on the screen (the output).

hello.py

```
print("Hello, world!")
print("Python is running.")
```

> **Tip**
> Press **Run** ▶ on any code box to execute it in your browser, or copy the code and paste it into the Sandbox page.

Variables store values

A **variable** is a name that points to a value. You create one with the assignment operator `=`. The right side is evaluated first, then the result is stored under the name on the left. Read `=` as "gets", not as "equals".

variables.py

```
name = "Ada"
age = 36
print(name)
print(age)
```

Trace: a variable is reassigned TRACE

```
1  x = 5
2  y = x + 2
3  x = 10
4  print(y)
```

Step	Line run	Variables after the line	Output so far
1	1	`x = 5`	`—`
2	2	`x = 5, y = 7`	`—`
3	3	`x = 10, y = 7`	`—`
4	4	`x = 10, y = 7`	`7`

Naming rules

- Names may use letters, digits, and underscores `_`.
- A name cannot start with a digit. `2nd_place` is illegal; `second_place` is fine.
- Names are **case-sensitive**. `Score` and `score` are two different variables.
- Names cannot be Python keywords such as `for`, `if`, `while`, or `True`.
- Use **snake_case** (lowercase words joined by underscores). It is the Python convention.

Name	Legal?	Why
`total_price`	Yes	Descriptive snake_case
`x1`	Yes	Letters then a digit is fine
`1st`	No	Starts with a digit
`my-name`	No	A hyphen is read as minus
`class`	No	Reserved keyword

Comments

A comment starts with `#`. Python ignores everything after it on that line. Use comments to explain *why* code does something, not to repeat what the code already says.

commented.py

```
# Store the sales tax rate so it is easy to change later.
TAX_RATE = 0.07

price = 40          # the price before tax
print(price * (1 + TAX_RATE))
```

Reading errors

When Python cannot run a line, it stops and prints an error. The last line says what went wrong, and the line above it says where. Errors are information, not failure.

broken.py

```
print(score)
```

That produces `NameError: name 'score' is not defined`. The name `score` was never assigned. Fix it by creating the variable first.

Predict the output THINK FIRST

Before you run anything, decide what each program prints.

```
print(2 + 3)
```

Answer: `5`

```
print("2 + 3")
```

Answer: `2 + 3`
Inside quotes, Python treats the text literally. It is a string, not math.

```
a = 1
b = a
a = 2
print(b)
```

Answer: `1`
`b` received the value of `a` at that moment. Later changes to `a` do not affect `b`.

```
print(score)
```

Answer: `NameError`
The name was never assigned, so Python cannot find it.

Practice

▼ **Exercise 1 — Introduce yourself**

Create three variables: `name` (text), `favorite_number` (a whole number), and `city` (text). Print each one on its own line.

▼ **Hint**

Text goes in quotes. Numbers do not.

▼ **Exercise 2 — Swap two values**

Given `a = 1` and `b = 2`, write code that swaps them so `a` ends with 2 and `b` ends with 1. Use a third variable called `temp`.

▼ **Hint**

Save one value before you overwrite it.

Discussion

1. Why do you think Python uses the `=` symbol for assignment when math uses it for equality? What does that cost a beginner?
2. Give an example of a comment that repeats the code and one that explains why. Which one is more useful six months from now?
3. Which naming rule do you think causes the most confusion for new programmers, and why?

Key terms

Statement

One complete instruction the interpreter runs, such as `print("hi")` or `x = 3`.

Interpreter

The program that reads Python code and runs it. Python code runs line by line through the interpreter.

Variable

A name that refers to a value stored in memory. The value can be changed by assigning a new one.

Assignment

Storing a value under a name using `=`. The right side is computed first.

print()

A built-in function that writes its arguments to the output, separated by spaces, followed by a newline.

Comment

Text after `#` that Python ignores. Used to explain intent to human readers.

Keyword

A word reserved by the language (for example `if`, `for`, `True`). It cannot be used as a variable name.

snake_case

Naming style with lowercase words joined by underscores, such as `total_price`. Python's convention for variables and functions.

NameError

The error raised when code uses a name that has not been assigned.

SyntaxError

The error raised when code breaks the grammar of the language, such as a missing quotation mark or parenthesis.

See all terms in the glossary →

UNIT 2 60 min

Data Types and Strings

Integers, floats, strings, and booleans; checking types; converting between them; working with text.

By the end of this unit you will be able to

- Identify the common types: `int`, `float`, `str`, `bool`
- Check a value's type with `type()` and convert with `int()`, `float()`, `str()`
- Explain why `0.1 + 0.2` is not exactly `0.3`
- Index and slice strings, and use common string methods
- Build readable output with f-strings

Every value has a type

A **data type** tells Python what kind of value something is and what operations make sense on it. Python checks types as the program runs.

Type	Name in code	Examples	What it holds
Integer	`int`	`42`, `-7`, `0`	Whole numbers
Float	`float`	`3.14`, `-0.5`, `2.0`	Numbers with a decimal point
String	`str`	`"Hello"`, `'a'`	Text, in quotes
Boolean	`bool`	`True`, `False`	Yes or no. Capitalized.

types.py

```
age = 15          # int
height = 5.8      # float
name = "Jordan"   # str
is_student = True # bool

print(type(age))
print(type(height))
print(type(name))
print(type(is_student))
```

Arithmetic and division

Python has two division operators. `/` always returns a float. `//` is floor division and returns a whole number. `%` is the remainder, which is useful for checking even and odd.

division.py

```
print(7 / 2)    # 3.5
print(7 // 2)   # 3
print(7 % 2)    # 1
print(2 ** 10)  # 1024
```

Converting between types

Input and file data often arrive as text. Convert with `int()`, `float()`, or `str()`. This is called **casting**.

casting.py

```
text = "42"
number = int(text)
print(number + 8)     # 50 (math)
print(text + "8")     # 428 (joins two strings)
print(float("3.5") * 2)
print(str(99) + " bottles")
```

Watch out

`int("3.7")` raises a `ValueError`. Convert to `float` first, then to `int` if you need to drop the decimal part: `int(float("3.7"))` gives `3`.

Floats are approximate

Computers store most decimal numbers in binary, which cannot represent 0.1 exactly. Small errors show up. Compare floats with a tolerance, not with `==`. Later you will use `math.isclose()`.

float_surprise.py

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(round(0.1 + 0.2, 2))
```

Strings

A string is an ordered sequence of characters. Each character has an **index**, starting at 0. Negative indexes count from the end. A **slice** `text[start:stop]` returns a part of the string; `stop` is not included.

strings.py

```
word = "Python"
print(word[0])      # P
print(word[-1])     # n
print(word[0:3])    # Pyt
print(word[2:])     # thon
print(len(word))    # 6
print(word.upper()) # PYTHON
print("py" in word.lower())  # True
```

Strings are **immutable**. You cannot change one character in place. You build a new string instead.

> **Watch out**
> `word[0] = "J"` raises `TypeError: 'str' object does not support item assignment`. Use `"J" + word[1:]` to build `"Jython"`.

Method	What it does	Example result
`.upper()` / `.lower()`	Changes letter case	`"hi".upper()` → `"HI"`
`.strip()`	Removes spaces at both ends	`"  hi  ".strip()` → `"hi"`
`.replace(old, new)`	Replaces every match	`"a-b".replace("-", "+")` → `"a+b"`
`.split(sep)`	Splits into a list	`"a,b".split(",")` → `["a", "b"]`
`.startswith(text)`	Checks the beginning	`"Python".startswith("Py")` → `True`

f-strings

Put an `f` before the opening quote and wrap any expression in braces. This is the cleanest way to mix text and values.

fstrings.py

```
score = 92
student = "Maya"
print(f"{student} scored {score}% on the test.")
print(f"Next goal: {score + 3}%")
```

Predict the output THINK FIRST

Work out each result by hand first.

```
print(7 / 2)
```

Answer: `3.5`

```
print(7 // 2)
```

Answer: `3`

```
print("5" * 3)
```

Answer: `555`
Multiplying a string repeats it.

```
print(bool(""))
```

Answer: `False`
An empty string is "falsy".

```
print(int("3.7"))
```

Answer: `ValueError`
A decimal string cannot be parsed as an int.

Practice

▼ **Exercise 1 — Full name**

Store a first name and a last name in two variables. Print the full name using an f-string, for example `Ada Lovelace`.

▼ **Exercise 2 — Clean and count**

Start with `raw = "  Python Is Fun  "`. Remove the extra spaces, make it lowercase, and print how many characters it has.

▼ **Hint**

Methods can be chained: `raw.strip().lower()`. Then call `len()` on the result.

Discussion

1. Why do you think `"5" + "5"` gives `"55"` while `5 + 5` gives `10`? What does that tell you about types?
2. Where in your daily life do you see numbers stored as text, and what would break if a program treated them as numbers?
3. If floats cannot represent 0.1 exactly, when should a bank program use integers (cents) instead?

Key terms

Data type

A category of value (int, float, str, bool) that determines what operations are valid on it.

int

Integer: a whole number with no decimal point.

float

Floating-point number: a number with a decimal point. Stored approximately.

str

String: an immutable sequence of characters written in quotes.

bool

Boolean: the value `True` or `False`. Named for George Boole.

type()

A built-in function that returns the type of a value, such as `<class 'int'>`.

Casting

Converting a value from one type to another, for example `int("42")`.

Index

The position of an item in a sequence. Counting starts at 0.

Slice

A part of a sequence written `seq[start:stop]`. The stop position is excluded.

Immutable

A value that cannot be changed after it is created. Strings, integers, and tuples are immutable.

f-string

A string literal beginning with `f` that embeds expressions in braces, such as `f"{x}"`.

Method

A function that belongs to an object and is called with a dot, such as `name.upper()`.

See all terms in the glossary →

UNIT 3 60 min

Boolean Logic and Truth Tables

Comparison operators, the logical operators and, or, not, precedence, and reading truth tables.

By the end of this unit you will be able to

- Write comparisons with `==`, `!=`, `<`, `>=` and explain the result
- Combine conditions with `and`, `or`, and `not`
- Build and read truth tables for a logic expression
- Explain short-circuit evaluation and why order matters
- Predict the result of an expression that mixes several operators

Comparisons produce booleans

A comparison asks a yes-or-no question and produces a `bool`. Every condition you will write in the next units is a comparison or a combination of them.

Operator	Meaning	Example	Result
`==`	equal to	`3 == 3`	`True`
`!=`	not equal to	`3 != 3`	`False`
`<` / `>`	less / greater than	`2 < 5`	`True`
`<=` / `>=`	less or equal / greater or equal	`5 >= 6`	`False`

Watch out

`=` assigns. `==` compares. Writing `if x = 5:` is a `SyntaxError`.

and, or, not

`and` is True only when **both** sides are True. `or` is True when **at least one** side is True. `not` flips the value.

logic.py

```
a = True
b = False
print(a and b)    # False
print(a or b)     # True
print(not a)      # False
print(5 > 3 and 2 == 2)   # True
```

Truth tables

A **truth table** lists every combination of inputs and the output for each. With two variables there are 2 × 2 = 4 rows. With three there are 8. Each extra variable doubles the rows. Use the table below to predict each column, then reveal the answers.

Truth tables for A and B INTERACTIVE

Predict each column first, then press **Reveal results**.

A	B	A and B	A or B	A != B (exclusive or)	not A
True	True	True	True	False	False
True	False	False	True	True	False
False	True	False	True	True	True
False	False	False	False	False	True

Note

The "exclusive or" column (called XOR) is True when exactly one input is True. It is the same as `!=` on two booleans.

Precedence

Operators run in a fixed order. From highest to lowest: comparisons, then `not`, then `and`, then `or`. Use parentheses whenever you are not sure. They make the code easier to read as well.

precedence.py

```
print(True or False and False)     # True: "and" runs first
print((True or False) and False)   # False
print(not True or True)            # True: "not" runs first
```

Short-circuit evaluation

Python stops evaluating as soon as the answer is known. In `x and y`, if `x` is False the whole expression is False and `y` is never run. In `x or y`, if `x` is True, `y` is skipped. This lets you write safe checks like `name is not None and name.strip() != ""`.

short_circuit.py

```
def check():
    print("check() was called")
    return True

print(False and check())   # check() never runs
print(True or check())    # check() never runs
```

Truthy and falsy values

Every value can be used as a condition. `0`, `0.0`, `""`, `[]`, and `None` are **falsy**. Everything else is **truthy**. Use this to write short checks like `if items:` to test for a non-empty list.

Predict the output THINK FIRST

Be careful with precedence and short-circuiting.

```
print(not 0)
```

Answer: `True`
0 is falsy, and `not` flips it.

```
print(3 == 3.0)
```

Answer: `True`
Int and float compare by value.

```
print("apple" < "banana")
```

Answer: `True`
Strings compare alphabetically, letter by letter.

```
print(1 < 2 < 3)
```

Answer: `True`
Python allows chained comparisons. It means `1 < 2 and 2 < 3`.

```
print(False and True or True)
```

Answer: `True`
`and` binds tighter, so it is `(False and True) or True`.

```
print(True or print("hi"))
```

Answer: `True`
The `or` already knows the answer, so `print("hi")` never runs.

Practice

▼ **Exercise 1 — Between 1 and 10**

Given `x = 7`, write one expression that is True only when `x` is from 1 to 10 inclusive. Print it.

▼ **Hint**

Python lets you chain comparisons. You can also write it with `and`.

▼ **Exercise 2 — NAND table**

Write a truth table for `not (A and B)`. Predict every row before you run the code, then check.

Discussion

1. Restaurants say "you may have soup or salad." Is that an inclusive or an exclusive or? How would you write each in Python?
2. Why is `a or b` safe to write when `a` is a check like `user is None`?
3. Can you write an expression equivalent to `A or B` using only `and` and `not`? Try it with a truth table.

Key terms

Boolean

A value that is either `True` or `False`.

Comparison operator

An operator that compares two values and returns a boolean: `==`, `!=`, `<`, `>`, `<=`, `>=`.

Logical operator

`and`, `or`, and `not`, which combine or invert boolean values.

Truth table

A table listing every combination of inputs and the resulting output of a logic expression.

XOR (exclusive or)

True when exactly one of two inputs is True. In Python, `a != b` on booleans.

Operator precedence

The fixed order in which operators are evaluated. Parentheses override it.

Short-circuit evaluation

Stopping a boolean expression as soon as its result is known, so later parts are never evaluated.

Truthy / falsy

Values that count as True or False in a condition. Falsy values include `0`, `""`, `[]`, and `None`.

Chained comparison

A comparison such as `1 < x < 10`, which Python evaluates as two comparisons joined by `and`.

See all terms in the glossary →

UNIT 4 60 min

Conditionals: Making Decisions

if, elif, and else; indentation as structure; nested decisions; the order of branches.

By the end of this unit you will be able to

- Write an `if` statement and explain why indentation matters
- Choose between a chain of `elif` branches and separate `if` statements
- Trace which branch runs for a given input
- Nest conditionals and keep them readable
- Avoid the common mistakes: `=` for `==`, missing colons, and wrong branch order

The if statement

An `if` statement runs a block of code only when its condition is True. The condition ends with a colon. The block under it is indented, usually by four spaces. Python uses indentation to know where the block ends.

if_basic.py

```
temperature = 92

if temperature > 85:
    print("Turn on the fan.")
    print("Drink some water.")

print("Done checking.")
```

The last line is not indented, so it runs no matter what the temperature is.

if, elif, else

Use `elif` ("else if") to test a list of alternatives. Python checks them in order and runs **only the first** branch that is True. Then it skips the rest. The `else` branch catches everything left over.

grades.py

```
score = 84

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print("Grade:", grade)
```

Trace: which branch runs? TRACE

```
score = 84

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print("Grade:", grade)
```

Step	Line run	Variables after the line	Output so far
1	1	`score = 84`	—
2	3	`score = 84`	—
3	5	`score = 84`	—
4	6	`score = 84, grade = 'B'`	—
5	12	`score = 84, grade = 'B'`	`Grade: B`

Watch out

Order matters. If you test `score >= 70` first, a score of 95 lands in the "C" branch. Put the strictest condition first, or use separate `if` statements.

Nested decisions

You can put a conditional inside another one. Each extra level adds four spaces. If you find yourself nesting three or more levels deep, consider a different structure, such as checking for the failing case first and returning or continuing early.

nested.py

```
age = 17
has_ticket = True

if has_ticket:
    if age >= 18:
        print("Enter, adult.")
    else:
        print("Enter, minor. Guardian required.")
else:
    print("Buy a ticket first.")
```

Combining conditions

Use `and`, `or`, and `not` from Unit 3 to combine tests into one condition. This is often cleaner than nesting.

combined.py

```
age = 17
has_ticket = True

if has_ticket and age >= 18:
    print("Enter, adult.")
elif has_ticket:
    print("Enter, minor.")
else:
    print("Buy a ticket first.")
```

Conditional expressions

For a simple choice between two values, Python has a one-line form: `value_if_true if condition else value_if_false`. Use it for short values only.

ternary.py

```
n = 7
print("even" if n % 2 == 0 else "odd")
```

Predict the output THINK FIRST

Predict which lines print. Pay attention to indentation and branch order.

```
x = 5
if x > 3:
    print("big")
```

Answer: `big`

```
x = 5
if x > 3:
    print("a")
if x > 1:
    print("b")
```

Answer: `a b`
Two separate `if` statements are both checked.

```
x = 5
if x > 3:
    print("a")
elif x > 1:
    print("b")
```

Answer: `a`
Only the first true branch runs.

```
x = 0
if x:
    print("yes")
else:
    print("no")
```

Answer: `no`
0 is falsy.

```
print("even" if 4 % 2 == 0 else "odd")
```

Answer: `even`

Practice

▼ **Exercise 1 — Adult or minor**

Given `age = 16`, print `Adult` if the age is 18 or more, otherwise print `Minor`.

▼ **Exercise 2 — Ticket price**

A movie ticket costs $12. Students (age 17 and under) pay $8. Seniors (age 65 and over) pay $9. Given `age = 70`, print the price.

▼ **Hint**

Check the age groups in an order that makes each branch correct.

Discussion

1. Rewrite the grades example with separate `if` statements instead of `elif`. What breaks, and why?
2. When is nesting clearer than combining with `and`? When is it worse?
3. Give an example of a real-world rule that has an `else` branch you would not want to forget.

Key terms

Condition
An expression that evaluates to True or False and decides whether a block runs.

Branch
One path through a conditional. Each `if`, `elif`, and `else` defines a branch.

Indentation
Leading spaces that show which lines belong to a block. Python requires consistent indentation.

Block
A group of statements that run together, introduced by a colon and indented beneath a statement.

elif
Short for "else if". Tests another condition only if the previous ones were False.

Nested conditional
A conditional placed inside another conditional's block.

Conditional expression
A one-line choice written `a if condition else b`.

See all terms in the glossary →

UNIT 5 70 min

Loops: Repeating Work

for loops over ranges, while loops with conditions, accumulators, break, and continue.

By the end of this unit you will be able to

- Write a `for` loop over a `range()` and explain what the loop variable holds
- Write a `while` loop and identify the update that makes it end
- Use an accumulator to build up a total
- Use `break` and `continue` to change the flow of a loop
- Recognize and avoid an infinite loop

Why loops

A loop repeats a block of code. Without loops you would copy a line 1,000 times. With them, the program repeats the same logic for each item, each number, or until a condition changes.

for loops and range()

A `for` loop visits each item in a sequence, one at a time. `range(stop)` produces the numbers from 0 up to but **not including** `stop`. `range(start, stop)` starts at `start`. `range(start, stop, step)` counts by `step`.

for_range.py

```
for i in range(3):
    print("Count:", i)

for n in range(2, 10, 3):
    print("Stepped:", n)
```

Trace: summing 1 to 3 TRACE

```
1  total = 0
2  for i in range(1, 4):
3      total = total + i
4  print(total)
```

Step	Line run	Variables after the line	Output so far
1	1	`total = 0`	`—`
2	2	`total = 0, i = 1`	`—`
3	3	`total = 1, i = 1`	`—`
4	2	`total = 1, i = 2`	`—`
5	3	`total = 3, i = 2`	`—`
6	2	`total = 3, i = 3`	`—`
7	3	`total = 6, i = 3`	`—`
8	2	`total = 6, i = 3`	`—`
9	4	`total = 6, i = 3`	`6`

Note

`range(1, 4)` gives 1, 2, 3. It does not include 4. That is the most common off-by-one mistake in this unit.

while loops

A `while` loop keeps running as long as its condition is True. It is checked before each pass. You must change something inside the loop, or it never stops.

countdown.py

```
count = 3
while count > 0:
    print(count)
    count = count - 1
print("Liftoff!")
```

Trace: the countdown TRACE

```
1  count = 3
2  while count > 0:
3      print(count)
4      count = count - 1
5  print("Liftoff!")
```

Step	Line run	Variables after the line	Output so far
1	1	`count = 3`	—
2	2	`count = 3`	—
3	3	`count = 3`	`3`
4	4	`count = 2`	`3`
5	2	`count = 2`	`3`
6	3	`count = 2`	`3 | 2`
7	4	`count = 1`	`3 | 2`
8	2	`count = 1`	`3 | 2`
9	3	`count = 1`	`3 | 2 | 1`
10	4	`count = 0`	`3 | 2 | 1`
11	2	`count = 0`	`3 | 2 | 1`
12	5	`count = 0`	`3 | 2 | 1 | Liftoff!`

Watch out

Infinite loops. If the condition never becomes False, the program runs forever. Common causes: forgetting to update the variable, or using `while True:` with no `break`. If you run one in this site, press the browser's reload button.

break and continue

- `break` leaves the loop immediately.
- `continue` skips the rest of this pass and moves to the next item.

break_continue.py

```
for n in range(1, 8):
    if n == 3:
        continue       # skip 3
    if n == 6:
        break          # stop before 6
    print(n)
```

Loops and conditions together

Loops and `if` statements combine freely. A common pattern is a loop that searches for something and stops when it finds it.

search.py

```
scores = [72, 95, 88, 61]
for s in scores:
    if s >= 90:
        print("First score at or above 90:", s)
        break
else:
    print("No score reached 90.")
```

Tip

The `else` on a loop runs only when the loop finished *without* hitting `break`. It is unusual, so label it clearly in your code.

Predict the output THINK FIRST

Trace each loop in your head before you run it.

```
for i in range(3):
    print(i)
```

Answer: `0 1 2`

`range(3)` stops before 3.

```
print(list(range(2, 10, 3)))
```

Answer: `[2, 5, 8]`

```
for i in range(5):
    if i == 2:
        break
    print(i)
```

Answer: `0 1`

```
for i in range(4):
    if i % 2 == 0:
        continue
    print(i)
```

Answer: `1 3`

```
n = 0
while n < 3:
    n = n + 1
    print(n)
```

Answer: `1 2 3`

```
print(sum(range(1, 5)))
```

Answer: `10`
1 + 2 + 3 + 4.

Practice

▼ **Exercise 1 — Divisible by three**

Print every number from 1 to 20 that is divisible by 3.

▼ **Hint**

Use `%`. A number is divisible by 3 when `n % 3 == 0`.

▼ **Exercise 2 — Running total**

Starting at `total = 0`, add 7 to `total` each time, using a `while` loop. Stop as soon as `total` is 50 or more. Print the final total.

Discussion

1. When would you choose `for` and when `while`? Give one example of each from everyday life.
2. The trace shows line 2 running one more time than the body. Why does the check need to run one extra time?
3. What is the risk of using `while True:`, and how does a `break` reduce it?

Key terms

Loop

A statement that repeats a block of code.

Iteration

One pass through the body of a loop.

for loop

A loop that assigns each item of a sequence to a variable in turn.

while loop

A loop that repeats while its condition is True. The condition is checked before each pass.

range()

A built-in that produces a sequence of integers from `start` up to but not including `stop`.

Accumulator

A variable that collects a running result, such as a total, across loop iterations.

break

A statement that exits the nearest loop immediately.

continue

A statement that skips the rest of the current iteration and moves to the next.

Infinite loop

A loop whose condition never becomes False, so it never ends unless interrupted.

Off-by-one error

A bug caused by a range or index that is one too high or too low.

See all terms in the glossary →

UNIT 6 70 min

Functions

Define reusable code with def, pass inputs, return results, and understand local versus global scope.

By the end of this unit you will be able to

- Define a function with `def` and call it
- Distinguish parameters from arguments
- Return a value with `return` and explain what happens when there is no `return`
- Use default and keyword arguments
- Explain why variables inside a function are local

Why functions

A **function** is a named block of code you can run as many times as you need. Functions make programs shorter, easier to test, and easier to change, because the logic lives in one place. You have been calling functions since Unit 1: `print()`, `range()`, and `int()` are all functions.

Defining and calling

greet.py

```
def greet(name):
    print(f"Hello, {name}!")

greet("Ada")
greet("Ben")
```

- `def` starts the definition. The name follows, then parentheses, then a colon.
- **Parameters** are the names listed in the definition (`name`).
- **Arguments** are the values you pass when calling (`"Ada"`).

Returning values

`return` sends a value back to the caller and ends the function. A function without a `return` gives back `None`, Python's "no value" marker. Use `return` when the caller needs the result; use `print()` when the goal is only to show something.

Trace: a function with a return value TRACE

```
1  def add_tax(price, rate=0.07):
2      tax = price * rate
3      return price + tax
4
5  total = add_tax(100)
6  print(total)
```

Step	Line run	Variables after the line	Output so far
1	1	`add_tax = function`	—
2	2	`add_tax = function, price = 100, rate = 0.07, tax = 7.0`	—
3	3	`add_tax = function, price = 100, rate = 0.07, tax = 7.0`	—
4	5	`add_tax = function, total = 107.0`	—
5	6	`add_tax = function, total = 107.0`	`107.0`

Default and keyword arguments

A parameter can have a default value. Callers may leave it out. You can also pass arguments by name, which makes calls easier to read.

defaults.py

```
def greet(name="friend", punctuation="!"):
    print(f"Hi {name}{punctuation}")

greet()
greet("Maya")
greet(punctuation="?", name="Sam")
```

Scope: where variables live

A variable created inside a function is **local**. It exists only during that call. A variable created at the top level of a file is **global**. A function can *read* a global, but assigning to a name inside a function creates a new local variable instead of changing the global one.

scope.py

```
total = 10

def try_to_change():
    total = 99          # creates a new local variable
    print("inside:", total)

try_to_change()
print("outside:", total)
```

Watch out
Using a local variable outside its function raises `NameError`. Return the value instead of relying on a shared variable.

Good function habits

- Give each function one clear job and a verb name: `calculate_tax`, not `stuff`.
- Add a short docstring, a string as the first line of the body, that says what the function does.
- Prefer passing values in and returning results over changing globals.

docstring.py

```
def is_even(n):
    """Return True if n is divisible by 2."""
    return n % 2 == 0

print(is_even(4))
print(is_even(7))
```

Predict the output THINK FIRST

Remember: a missing `return` gives `None`.

```
def double(x):
    return x * 2

print(double(4))
```

Answer: `8`

```
def hi(name="friend"):
    print("Hi", name)

hi()
```

Answer: `Hi friend`

```
n = 5
def bump(n):
    n = n + 1
    return n

bump(n)
print(n)
```

Answer: `5`
The function returned 6 but nothing kept it, and the global `n` was never changed.

```
def check(x):
    if x > 0:
        return "positive"

print(check(-1))
```

Answer: `None`
No `return` ran, so the function gives back `None`.

Practice

▼ **Exercise 1 — Celsius to Fahrenheit**

Write a function `c_to_f(celsius)` that returns the Fahrenheit value (`celsius * 9 / 5 + 32`). Print the result for 100 and for 0.

▼ **Exercise 2 — Largest of three**

Write `largest(a, b, c)` that returns the biggest of the three numbers **without** using the built-in `max()`. Test it with 4, 9, and 2.

▼ **Hint**

Start by assuming `a` is the biggest, then compare it to `b` and `c` with `if` statements.

Discussion

1. Why does `scope.py` print 99 inside and 10 outside? What would make a beginner expect the outside value to change?
2. When would you want a default argument, and when would you rather require the caller to supply the value?
3. If you wrote one function that does three jobs, how would you split it?

Key terms

Function

A named, reusable block of code, defined with `def` and run by calling it.

Parameter

A variable name in a function definition that receives a value when the function is called.

Argument

The actual value passed to a function when it is called.

return

A statement that ends a function and sends a value back to the caller.

None

Python's special value meaning "no value". Functions without a `return` produce it.

Default argument

A parameter with a preset value, such as `rate=0.07`, used when the caller does not supply one.

Keyword argument

An argument passed by name, such as `greet(name="Sam")`.

Local variable

A variable that exists only inside the function where it was created.

Global variable

A variable defined at the top level of a program, visible to the code that follows it.

Docstring

A string literal placed as the first line of a function body to document what it does.

See all terms in the glossary →

UNIT 7 70 min

Lists

Ordered collections: indexing, slicing, changing and growing a list, sorting, and the aliasing trap.

By the end of this unit you will be able to

- Create lists and access items by index, including negative indexes
- Slice a list and explain what the slice includes
- Add, remove, and change items with `append`, `insert`, `remove`, `pop`, and assignment
- Explain why a list is mutable and what aliasing means
- Loop over a list and build a new list from it

What a list is

A **list** is an ordered collection of values, written in square brackets and separated by commas. The items can be any type, and they can be mixed. Lists are **mutable**: you can change them after they are created.

lists_basics.py

```
scores = [72, 95, 88, 61]
names = ["Ana", "Ben", "Cy"]
mixed = [1, "two", 3.0, True]

print(scores)
print(len(names))
print(type(mixed))
```

Indexing and slicing

Indexes start at 0. `list[-1]` is the last item. A slice `list[start:stop]` returns a new list that includes `start` and excludes `stop`.

indexing.py

```
letters = ["a", "b", "c", "d", "e"]
print(letters[0])     # a
print(letters[-1])    # e
print(letters[1:4])   # ['b', 'c', 'd']
print(letters[:2])    # first two
print(letters[3:])    # from index 3 to the end
```

Watch out

Asking for an index that does not exist raises `IndexError: list index out of range`. For a list of 5 items, the valid indexes are 0 to 4 (or -5 to -1).

Changing a list

Action	Code	Effect
Add to the end	`items.append(x)`	Adds one item at the end
Insert at a position	`items.insert(0, x)`	Puts x at index 0; everything shifts right
Remove by value	`items.remove(x)`	Removes the first match; error if absent
Remove by position	`items.pop(i)`	Removes and returns the item at i (default: last)
Replace	`items[2] = x`	Overwrites the item at index 2

changing.py

```
fruit = ["apple", "banana"]
fruit.append("cherry")
fruit.insert(0, "avocado")
fruit[1] = "blueberry"
removed = fruit.pop()
print(fruit)
print("removed:", removed)
```

Methods like `append` change the list in place and return `None`. Do not write `x = x.append(1)`. You would overwrite the list with `None`.

Sorting

`sorted(items)` returns a new sorted list and leaves the original alone. `items.sort()` sorts in place and returns `None`. Add `reverse=True` for descending order.

Aliasing: two names, one list

Assigning a list to a new name does **not** copy it. Both names point to the same list. Changing one changes the other. Use `.copy()` or `list(original)` when you need an independent copy.

Trace: aliasing TRACE

```
1  a = [1, 2]
2  b = a
3  b.append(3)
4  print(a)
```

Step	Line run	Variables after the line	Output so far
1	1	`a = [1, 2]`	—
2	2	`a = [1, 2], b = [1, 2]`	—
3	3	`a = [1, 2, 3], b = [1, 2, 3]`	—
4	4	`a = [1, 2, 3], b = [1, 2, 3]`	`[1, 2, 3]`

Looping over a list

A `for` loop visits each item in order. Use `enumerate()` when you also need the index.

loop_lists.py

```
names = ["Ana", "Ben", "Cy"]
for name in names:
    print("Hi", name)

for i, name in enumerate(names):
    print(i, name)
```

Tip

Do not add or remove items from a list while looping over it. Build a new list instead: `evens = [n for n in nums if n % 2 == 0]` is the compact form of a loop with `append`.

Predict the output THINK FIRST

Watch for in-place methods and aliasing.

```
nums = [10, 20, 30]
print(nums[1])
```

Answer: `20`

```
nums = [10, 20, 30]
print(nums[-1])
```

Answer: `30`

```
nums = [10, 20, 30]
nums.append(40)
print(len(nums))
```

Answer: `4`

```
print([1, 2, 3, 4, 5][1:4])
```

Answer: `[2, 3, 4]`

```
letters = ["a", "b"]
letters.insert(0, "z")
print(letters)
```

Answer: `['z', 'a', 'b']`

```
x = [3, 1, 2]
print(x.sort())
```

Answer: `None`
`sort()` changes the list and returns nothing.

```
print(5 in [1, 2, 3])
```

Answer: `False`

```
nums = [1, 2, 3]
nums[5] = 9
```

Answer: `IndexError`

Practice

▼ **Exercise 1 — Average score**

Given `scores = [72, 95, 88, 61]`, print the average. Use `sum()` and `len()`.

▼ **Exercise 2 — Squares**

Build a list called `squares` containing the squares of 1 through 5 (1, 4, 9, 16, 25). Use a loop and `append`. Print the list.

Discussion

1. Why do you think Python makes lists mutable instead of returning a new list for every change? What does that cost in clarity?
2. Predict: if two friends share one shopping list in a file and one of them deletes an item, what happens to the other? How is that like aliasing?
3. When is `sorted()` the better choice than `sort()`?

Key terms

List

An ordered, mutable collection of values written in square brackets.

Index

The position of an item in a list, starting at 0. Negative indexes count from the end.

Slice

A new list made from a range of positions, written `items[start:stop]`.

Mutable

A value that can be changed after it is created. Lists and dictionaries are mutable.

Method

A function attached to an object. `append`, `insert`, `pop`, and `sort` are list methods.

Aliasing

When two names refer to the same object, so a change through one name is visible through the other.

enumerate()

A built-in that gives each item of a sequence together with its index, for use in a loop.

IndexError

The error raised when an index is outside the range of a list.

List comprehension

A compact expression that builds a list from a loop, such as `[n * n for n in range(5)]`.

See all terms in the glossary →

UNIT 8 60 min

Tuples and Sets

Fixed-size tuples for grouped values, and sets for unique items with fast membership tests.

By the end of this unit you will be able to

- Create tuples and explain why they are immutable
- Unpack a tuple into separate variables and return several values from a function
- Create sets, explain why they hold no duplicates, and use union, intersection, and difference
- Choose between a list, a tuple, and a set for a task

Tuples: fixed groups

A **tuple** is an ordered collection written with parentheses. Like a list, it is indexed and sliced. Unlike a list, it is **immutable**: once created, you cannot add, remove, or replace items. Use a tuple when the group should not change, such as an (x, y) coordinate or an RGB color.

tuples.py

```
point = (4, 7)
color = (255, 128, 0)
print(point[0])
print(len(color))
```

Watch out

A single-item tuple needs a trailing comma: `(5,)` is a tuple, but `(5)` is just the number 5.

Unpacking

You can assign each item of a tuple to its own name in one line. This is called **unpacking**. The number of names must match the number of items.

unpacking.py

```
x, y = (3, 4)
print(x + y)

def min_max(nums):
    return min(nums), max(nums)

low, high = min_max([7, 2, 9, 4])
print(low, high)
```

Python functions return multiple values by returning a tuple. The caller unpacks it. This is the most common reason you will see tuples in real code.

Sets: unique items

A **set** is an unordered collection of unique values, written with curly braces. Adding a duplicate does nothing. Checking whether an item is in a set is very fast, even for large sets.

sets.py

```
visited = {1, 2, 2, 3, 3, 3}
print(visited)
print(len(visited))
visited.add(4)
print(4 in visited)
```

> **Watch out**
> `{}` creates an empty **dictionary**, not a set. Use `set()` for an empty set. Also, the order of items in a set is not guaranteed, so do not rely on it.

Set operations

Operation	Symbol	Method	Meaning
Union	`a | b`	`a.union(b)`	Items in either set

Operation	Symbol	Method	Meaning
Intersection	`a & b`	`a.intersection(b)`	Items in both sets
Difference	`a - b`	`a.difference(b)`	Items in a but not in b

set_ops.py

```
math_class = {"Ana", "Ben", "Cy"}
art_class = {"Ben", "Dee"}

print(math_class | art_class)
print(math_class & art_class)
print(math_class - art_class)
```

Choosing a collection

Need	Use	Why
An ordered sequence that changes	list	Indexable, mutable, allows duplicates
A fixed group, such as coordinates	tuple	Immutable; safe to pass around
Unique items, fast membership tests	set	Duplicates removed automatically
Labels that map to values	dict (Unit 9)	Look up by key instead of position

Tip

To remove duplicates from a list while keeping it a list: `unique = list(set(items))`. The order may change, so use this only when order does not matter.

Predict the output THINK FIRST

Watch for the single-item tuple and for the empty set.

```
print((5))
```

Answer: `5`
Parentheses alone do not make a tuple.

```
print((5,))
```

Answer: `(5,)`

```
t = (1, 2)
t[0] = 9
```

Answer: `TypeError`
Tuples cannot be changed.

```
x, y = (3, 4)
print(x + y)
```

Answer: `7`

```
print({1, 2, 2, 3})
```

Answer: `{1, 2, 3}`

```
print({1, 2} | {2, 3})
```

Answer: `{1, 2, 3}`

```
print({1, 2, 3} & {2, 3, 4})
```

Answer: `{2, 3}`

```
print(len(set("banana")))
```

Answer: `3`
The unique letters are b, a, n.

```
print({} == set())
```

Answer: `False`
`{}` is an empty dictionary.

Practice

▼ **Exercise 1 — Shared clubs**

Two clubs have these members: `chess = ["Ana", "Ben", "Cy", "Ben"]` and `robotics = ["Ben", "Dee", "Ana"]`. Print the names that appear in **both** clubs, without duplicates.

▼ **Hint**

Convert each list to a set, then use `&`.

▼ **Exercise 2 — Unpack a return**

Write a function `stats(nums)` that returns a tuple of `(smallest, largest, count)`. Call it with `[4, 8, 1, 6]` and unpack the result into three variables. Print each.

Discussion

1. Why might a programmer choose a tuple over a list even when both would work? What is the benefit of "can't change it"?
2. A set throws away duplicates and order. When is that a feature, and when is it a bug?
3. Pick one real data source (a roster, a list of coordinates, a log of website visits) and say which collection type you would use and why.

Key terms

Tuple
An immutable, ordered collection written in parentheses, such as `(4, 7)`.

Immutable
Cannot be changed after creation. Tuples, strings, and numbers are immutable.

Unpacking
Assigning each item of a sequence to a separate name in one statement, such as `x, y = point`.

Set
An unordered collection of unique values, written in curly braces or created with `set()`.

Union
The set of items in either of two sets. Written `a | b`.

Intersection
The set of items in both sets. Written `a & b`.

Difference
The items in the first set that are not in the second. Written `a - b`.

Membership test
Checking whether a value is in a collection with the `in` operator.

See all terms in the glossary →

UNIT 9 75 min

Dictionaries

Store values under named keys: create, read, update, safely look up, loop over, and nest dictionaries.

By the end of this unit you will be able to

- Create a dictionary and read, add, and update values by key
- Use `.get()` to look up a key that might be missing
- Loop over keys, values, and key-value pairs with `.items()`
- Build a dictionary by counting items in a loop
- Work with a dictionary nested inside another dictionary

Key-value pairs

A **dictionary** maps **keys** to **values**. Instead of remembering that item 2 is Ben's score, you look up `scores["Ben"]`. Keys must be immutable (strings and numbers are the usual choices). Values can be anything.

dict_basics.py

```
student = {
    "name": "Maya",
    "age": 16,
    "grades": [91, 88, 95]
}

print(student["name"])
print(student["grades"][0])
```

Adding and changing values

Assigning to a key that exists updates it. Assigning to a new key adds it.

dict_update.py

```
scores = {"Ana": 90}
scores["Ben"] = 85      # new key: added
scores["Ana"] = 95      # existing key: updated
print(scores)
```

Safe lookups with get()

Reading a missing key raises `KeyError`. `.get(key, default)` returns the default instead, which is the safe choice when the key might not exist.

get_safe.py

```
prices = {"apple": 0.50, "pear": 0.75}
print(prices.get("apple", 0))
print(prices.get("kiwi", 0))
```

> **Watch out**
> A missing key with `prices["kiwi"]` raises `KeyError: 'kiwi'`. Use `.get()` when the key is not guaranteed to exist.

Looping over a dictionary

`.keys()` gives the keys, `.values()` gives the values, and `.items()` gives both as pairs. Looping over the dictionary directly visits the keys.

Trace: loop over items TRACE

```
grades = {"Ana": 90}
grades["Ben"] = 85
grades["Ana"] = 95
for name, score in grades.items():
    print(name, score)
```

Step	Line run	Variables after the line	Output so far
1	1	`grades = {'Ana': 90}`	—
2	2	`grades = {'Ana': 90, 'Ben': 85}`	—
3	3	`grades = {'Ana': 95, 'Ben': 85}`	—
4	4	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ana', score = 95`	—
5	5	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ana', score = 95`	`Ana 95`
6	4	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ben', score = 85`	`Ana 95`
7	5	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ben', score = 85`	`Ana 95 | Ben 85`

Counting with a dictionary

A very common pattern is counting how often each item appears. Start empty, then for each item, use `.get(item, 0) + 1` to add one to its count.

counter.py

```
words = "the cat saw the dog and the bird".split()
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
print(counts)
print("'the' appears", counts["the"], "times")
```

Nested dictionaries

A dictionary can hold other dictionaries or lists. Chain the keys to reach deeper values. Real data (JSON from web APIs) looks exactly like this.

nested.py

```
class_roster = {
    "period_1": {"teacher": "Ms. Lee", "students": ["Ana", "Ben"]},
    "period_2": {"teacher": "Mr. Cruz", "students": ["Cy", "Dee", "Eli"]}
}

print(class_roster["period_2"]["teacher"])
print(len(class_roster["period_2"]["students"]))
```

Predict the output THINK FIRST

Keys decide what is found. Watch for duplicates and for missing keys.

```
d = {"a": 1}
print(d["a"])
```

Answer: `1`

```
d = {"a": 1}
print(d["b"])
```

Answer: `KeyError`

```
print({"x": 1}.get("y", 0))
```

Answer: `0`

```
d = {"k": 2}
d["k"] = 5
print(d)
```

Answer: `{'k': 5}`

```
print(len({"a": 1, "a": 2}))
```

Answer: `1`

The second "a" replaces the first. Keys are unique.

```
print(1 in {"a": 1})
```

Answer: `False`
`in` checks keys, not values.

```
d = {"p": {"age": 3}}
print(d["p"]["age"])
```

Answer: `3`

Practice

▼ **Exercise 1 — Inventory**

Start with `stock = {"pencil": 4, "pen": 2}`. Add 3 pens, add 5 erasers (a new key), then print the whole dictionary.

▼ **Exercise 2 — Most common**

Using the counts from the counter example, find the word with the highest count and print it. Test with the sentence `"a b a c a b"`.

▼ **Hint**

Loop over `counts.items()` and keep track of the best word and count seen so far.

Discussion

1. When is a list of pairs a better choice than a dictionary, and when is it worse?
2. Why do you think dictionary keys must be immutable? What would go wrong if a key could change after it was stored?
3. Think of a real system you use (a school portal, a game, a store app). What are its keys and values?

Key terms

Dictionary

A collection that maps unique keys to values, written in curly braces with `key: value` pairs.

Key

The label used to look up a value in a dictionary. Keys must be immutable and unique.

Value

The data stored under a key. Values can be any type.

.get()

A dictionary method that returns the value for a key, or a default if the key is missing.

.items()

A dictionary method that yields key-value pairs, usually unpacked in a loop as `for k, v in d.items()`.

KeyError

The error raised when reading a key that is not in the dictionary.

Nested dictionary

A dictionary that contains another dictionary or list as a value.

Counter pattern

Using a dictionary to count occurrences: `counts[item] = counts.get(item, 0) + 1`.

See all terms in the glossary →

UNIT 10 60 min

Libraries and Modules

Import the standard library, read its documentation, and split your own code into modules.

By the end of this unit you will be able to

- Import a module and call its functions with dot notation
- Use `from ... import` and `import ... as` and explain when each is appropriate
- Find a function's documentation with `help()` and `dir()`
- Explain what the standard library includes and what `pip` adds
- Create a module file and import it from another file

Modules and libraries

A **module** is a file of Python code that can be imported. A **library** is a collection of modules. Python ships with a large **standard library**, so many common tasks need no installation. Anything else you install with `pip`, which is covered in the NumPy and pandas units.

Importing

imports.py

```
import math
import statistics

print(math.sqrt(16))
print(math.pi)
print(math.floor(3.7))
print(statistics.mean([2, 4, 6]))
```

After `import math`, everything in the module is reached with a dot: `math.sqrt`. The name `math` comes first so you always know where a function lives.

import_styles.py

```
from math import ceil, factorial
print(ceil(2.1))
print(factorial(5))

import math as m
print(m.hypot(3, 4))
```

Tip

Use `import math` when many names come from one module. Use `from math import ceil` when you need only one or two names. An alias such as `as m` is common for long names; pandas and NumPy use `pd` and `np` by convention.

Finding out what a module offers

discover.py

```
import math

print(len(dir(math)))          # how many names the module has
print(math.sqrt.__doc__)       # the documentation string
```

In an interactive environment, `help(math.sqrt)` prints the same documentation. Reading the documentation is a skill you will use every day. Get in the habit of checking the docs before guessing.

Some useful standard-library tools

Module	What it gives you	Example
`math`	Square root, rounding, trig, constants	`math.sqrt(9)` → 3.0
`statistics`	Mean, median, standard deviation	`statistics.median([1, 3, 9])` → 3
`random`	Random choices and numbers	`random.choice(["a", "b"])`

Module	What it gives you	Example
`datetime`	Dates and times	`datetime.date.today()`
`csv` / `json`	Read and write common data files	`json.loads(text)`

> **Note**
> Random results differ on every run. To get the same "random" numbers each time while you test, call `random.seed(42)` first. Seeding is standard practice in data science and machine learning.

Writing your own module

Any `.py` file is a module. Put a helpful function in its own file and import it from your main program. Two files are needed, so this example is shown as code to save on your computer rather than in the browser sandbox.

greetings.py (module file)

```
def hello(name):
    """Return a friendly greeting."""
    return f"Hello, {name}!"

if __name__ == "__main__":
    # Runs only when you run this file directly, not when it is imported.
    print(hello("test"))
```

main.py (in the same folder)

```
from greetings import hello

print(hello("Ada"))
```

The `if __name__ == "__main__":` line lets a file work both as a script and as a module. Python sets `__name__` to `"__main__"` only when the file is run directly.

Watch out

Do not name your file the same as a standard-library module (for example `random.py` or `math.py`). Python will import your file instead of the library and the program will fail in confusing ways.

Predict the output THINK FIRST

Remember what `round()` does with exactly .5, and that `math` functions return floats.

```
import math
print(math.floor(-2.5))
```

Answer: `-3`

```
import math
print(math.ceil(2.1))
```

Answer: `3`

```
from math import pow
print(pow(2, 3))
```

Answer: `8.0`

```
import math
print(math.factorial(5))
```

Answer: `120`

```
print(round(2.5))
```

Answer: `2`
Python rounds .5 to the nearest even number ("banker's rounding").

```
print(round(3.5))
```

Answer: `4`

```
import math
print(math.hypot(3, 4))
```

Answer: `5.0`

```
print(abs(-7))
```

Answer: `7`

Practice

▼ **Exercise 1 — Hypotenuse two ways**

For a right triangle with legs 5 and 12, compute the hypotenuse once with `math.sqrt` and once with `math.hypot`. Print both results.

▼ **Exercise 2 — Build a module**

Create `shapes.py` with a function `circle_area(radius)` that returns `math.pi * radius ** 2`. In a second file, import it and print the area of a circle with radius 2. Write both files on your computer.

▼ **Hint**

The module file needs `import math` at the top.

Discussion

1. Why do you think Python includes a large standard library instead of making everyone install packages? What are the trade-offs?
2. What is the risk of a file named `random.py` in your project folder? How would you find the cause of the error?
3. When you see `import numpy as np` in someone else's code, what do you expect the rest of the code to look like?

Key terms

Module

A Python file (`.py`) whose functions and variables can be imported into another program.

Library

A collection of modules that provides a set of related tools, such as the standard library, NumPy, or pandas.

Standard library

The modules that come with every Python installation, such as `math`, `random`, and `json`.

import

A statement that loads a module so its names can be used.

Alias

A shorter name given to a module on import, such as `import numpy as np`.

pip

The package installer for Python. It downloads third-party libraries from the Python Package Index.

Seed

A starting value for a random number generator. The same seed produces the same sequence every run.

__name__

A built-in variable. It equals `"__main__"` when a file is run directly, and the module name when imported.

See all terms in the glossary →